

Tamper-Evidence Hashes — Implementation Summary

zord-intent-engine · feature/yashy/tamperEnhancement · 2026-07-14

Every hash below is `SHA-256(JCS_Canonicalize({ hash_type, hash_version, ...fields })))` unless noted otherwise. `JCS_Canonicalize` is our own `internal/jcs` package — sorted object keys (Go's `encoding/json` already sorts `map[string]any` keys), no insignificant whitespace, no HTML-escaping. This is equivalent to RFC 8785 output for the flat, ASCII-keyed objects we hash here. No external JCS library was added; `internal/jcs` has no internal dependencies, so both `internal/canonicalizer` and `internal/models` can import it without a cycle.

The 9 hashes

#	Hash	Formula fields	Computed in	Stored in
3	<code>canonical_row_hash</code>	<code>source_row_ref</code> , <code>client_payout_ref</code> , <code>beneficiary_fingerprint</code> , <code>amount_minor</code> , <code>currency</code> , <code>intended_execution_at</code> , <code>payment_rail</code> , <code>invoice_ref</code>	<code>computeCanonicalRowHash</code>	<code>payment_intents.canonical_row_hash</code> , <code>outbox.canonical_row_hash</code>
4	<code>tokenized_data_hash</code>	<code>tenant_id</code> , <code>beneficiary_name_token</code> , <code>account_number_token</code> , <code>ifsc_token</code> , <code>vpa_token</code> , <code>email_token</code> , <code>phone_token</code> , <code>tokenization_policy_version</code> — HMAC-SHA256 , not plain SHA-256	<code>computeTokenizedDataHash</code>	<code>payment_intents.tokenized_data_hash</code> , <code>outbox.tokenized_data_hash</code>
5	<code>business_idempotency_hash</code> (+ fallback)	preferred: <code>tenant_id</code> , <code>source_system</code> , <code>client_payout_ref</code> , <code>amount_minor</code> , <code>currency</code> . fallback: <code>tenant_id</code> , <code>beneficiary_fingerprint</code> , <code>amount_minor</code> , <code>currency</code> , <code>execution_date</code> , <code>invoice_ref</code> , <code>purpose_code</code>	<code>computeBusinessIdempotencyKey</code>	<code>payment_intents.business_idempotency_key</code> , <code>outbox.business_idempotency_key</code>
6	<code>mapping_profile_hash</code>	<code>profile_id</code> , <code>profile_version</code> , <code>source_system</code> , <code>detected_format</code> , <code>field_mappings</code> , <code>required_fields</code> (sorted), <code>soft_inferable_fields</code> (sorted), <code>field_kind_policy</code> , <code>sensitive_field_policy</code> , <code>normalization_rules</code> , <code>synonym_profile_version</code>	<code>MappingProfile.ComputeProfileHash</code> / <code>computeGenericMappingProfileHash</code> (fallback)	<code>payment_intents.mapping_profile_hash</code> , <code>outbox.mapping_profile_hash</code> , <code>mapping_profiles.profile_hash</code>
7	<code>canonical_payload_manifest_hash</code>	<code>manifest_schema_version</code> , <code>artifact_id</code> , <code>artifact_version_id</code> , <code>payload_type</code> , <code>canonicalization_version</code> , <code>mapping_profile_hash</code> , <code>row_count</code> , <code>rows</code> [[<code>source_row_ref</code> , <code>canonical_row_hash</code> , <code>amount_minor</code> , <code>currency</code> , <code>business_reference</code>]]	<code>ComputeFileManifestHash</code> , called from <code>UpdateBatchAggregateConfidence</code>	<code>canonical_batches.file_manifest_hash</code>
8	<code>raw_row_leaf_hash</code>	<code>tenant_id</code> , <code>artifact_id</code> , <code>artifact_version_id</code> , <code>source_row_ref</code> , <code>row_index</code> , <code>raw_row_hash</code>	<code>computeEvidenceLeafHashes</code>	<code>payment_intents.raw_row_evidence_leaf_hash</code> , <code>outbox.raw_row_evidence_leaf_hash</code>
9	<code>canonical_row_leaf_hash</code>	<code>tenant_id</code> , <code>artifact_id</code> , <code>artifact_version_id</code> , <code>source_row_ref</code> , <code>canonical_row_hash</code>	<code>computeEvidenceLeafHashes</code>	<code>payment_intents.canonical_row_evidence_leaf_hash</code> , <code>outbox.canonical_row_evidence_leaf_hash</code>
—	<code>input_facts_hash</code> (governance)	<code>amount_minor</code> , <code>currency</code> , <code>beneficiary_fingerprint</code> , <code>payment_rail</code> , <code>purpose_code</code> , <code>beneficiary_changed</code> , <code>is_possible_duplicate</code> , <code>daily_total_minor</code> , <code>previous_payment_count</code>	<code>computeGovernanceHash</code>	<code>payment_intents.input_facts_hash</code> , <code>outbox.input_facts_hash</code>
—	<code>governance_decision_hash</code>	<code>tenant_id</code> , <code>canonical_intent_hash</code> , <code>input_facts_hash</code> , <code>decision</code> , <code>reason_codes</code> , <code>required_approval_level</code> , <code>risk_level</code> — policy_id/policy_version/policy_hash deliberately excluded (no real policy engine yet)	<code>computeGovernanceHash</code> + <code>recomputeGovernanceDecisionHash</code>	<code>payment_intents.governance_hash</code> , <code>outbox.governance_hash</code>

Rows 3 / 7 / governance_decision were **existing fields we touched in place** (same column, new formula). Everything else is **new** — additive columns, nothing else disturbed.

Notable design points

- governance_decision_hash is computed twice.** At construction time `canonical_intent_hash` is still "" (the WORM chain hash doesn't exist until after the S3 snapshot step). Once `canonical_hash` is known, `recomputeGovernanceDecisionHash` re-derives it and `UpdateSnapshotRefs` persists both in the same round-trip.

- **canonical_payload_manifest_hash** rows are sorted by `source_row_ref`, then `business_reference`, then `canonical_row_hash` — not by insertion/ `source_row_num`, since row order has no business meaning for the manifest.
- **mapping_profile_hash** is never left blank. If no registered/global profile resolves for a request, `computeGenericMappingProfileHash` builds the hash from the NIR's actual per-field `source_path`s instead. Separately, `SeedGlobalMappingProfilesFromFile` now upserts `global_profiles.json` (TALLY, SAP, ...) into `mapping_profiles` at startup, so those built-ins resolve as real DB-backed profiles with a persisted hash instead of only an in-memory fallback.
- **tokenized_data_hash's key is derived, not stored.** `HMAC-SHA256(TOKENIZED_DATA_HASH_MASTER_SECRET, tenant_id)` gives a per-tenant key from one master secret (new env var, added to `.env` for local dev — set the real value via your secrets manager for other environments).
- **Placeholders, populated by upstream services later, not derived here:** `source_row_ref`, `invoice_ref`, `artifact_id`, `artifact_version_id` (`row_index` instead uses the already-derived `source_row_num`). `beneficiary_changed` and `previous_payment_count` also have no upstream signal yet and are hard-coded `false / 0`.
- **Two formulas change stored values for new rows going forward:** `business_idempotency_key` and `mapping_profile_hash` now use the new formulas per instruction to replace them in place — old DB rows keep their old-formula values until re-touched.

Files touched

- `internal/jcs/jcs.go` — shared canonicalizer + SHA-256/HMAC helpers (new)
- `internal/canonicalizer/row_hash.go` — `canonical_row_hash`, manifest hash (rewritten)
- `internal/canonicalizer/governance_hash.go` — `input_facts_hash`, `governance_decision_hash` (new)
- `internal/canonicalizer/business_idempotency_hash.go` — preferred + fallback (new)
- `internal/canonicalizer/tokenized_data_hash.go` — HMAC + key derivation (new)
- `internal/canonicalizer/evidence_leaf_hash.go` — raw/canonical row leaf hashes (new)
- `internal/models/mapping_profile.go` — `ComputeProfileHash` (rewritten)
- `internal/services/intent_service.go` — all wiring, `computeGenericMappingProfileHash`
- `internal/services/profile_resolver.go` — `SeedGlobalMappingProfilesFromFile`
- `internal/persistence/payment_intent_repo.go` — INSERT column lists, `UpdateBatchAggregateConfidence`, `UpdateSnapshotRefs`
- Migrations: `20260714090000_add_canonical_row_hash_and_file_manifest_hash.sql`, `20260714100000_add_row_hash_to_outbox.sql`, `20260714110000_add_governance_tokenized_evidence_hashes.sql`